

Naiyo24 Transfer File Sharing System

Professional architecture overview for the Flutter client, FastAPI backend, object-storage layer, database model, background jobs, deployment topology, and current implementation boundaries found in the repository.

GENERATED
June 8, 2026

REPOSITORY
naiyo24-file-sharing-system

SCOPE
Source inspection and architecture documentation

Executive Summary

The application is a cross-platform file-transfer experience. The visible Flutter client authenticates users, lets them select a file, chooses a short expiry window, uploads through the API, and returns a copyable share link. The backend is a Python FastAPI service that separates authentication, upload, share, and download responsibilities behind typed Pydantic schemas and SQLAlchemy data models.

Key Facts

Core pattern: Metadata in PostgreSQL, binaries in S3-compatible storage, transient upload state in Redis.

Primary stack: Flutter, Riverpod, Dio, FastAPI, SQLAlchemy, PostgreSQL, Redis, Celery, MinIO/S3, Nginx.

Current client path: The Flutter UI calls `/api/upload/simple`; chunked upload endpoints are implemented on the backend.

Contents

System architecture diagram; technology stack; client and backend layers; primary workflows; data model and storage; security, reliability, operations; implementation notes and source map.

System Architecture Diagram

Logical runtime view of the deployed application and request/data flow.

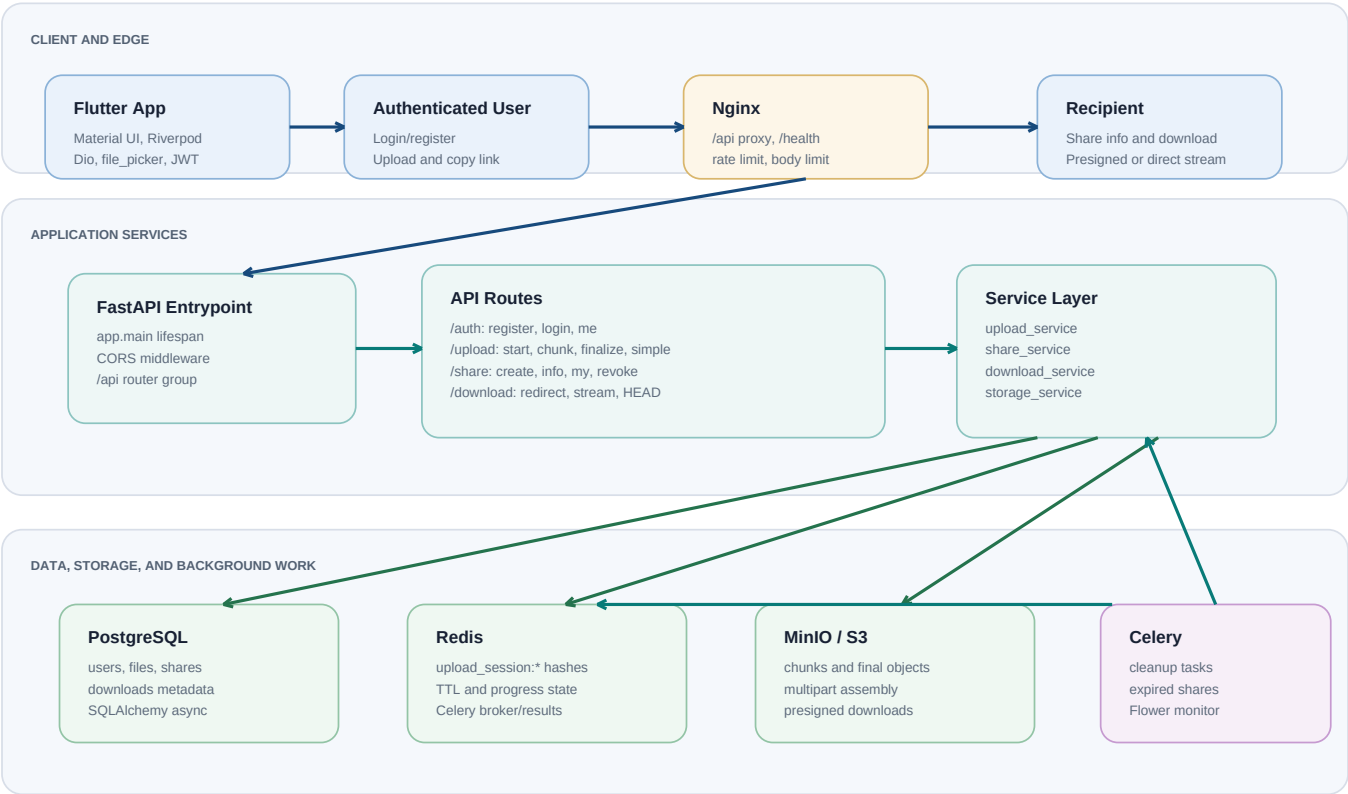


Diagram describes the repository implementation: Docker Compose runs the backend, PostgreSQL, Redis, MinIO, Nginx, Celery worker, and Flower. The Flutter client is configured separately and reaches the backend through its API base URL.

Client and users	Edge gateway	Application service	Persistence and storage
------------------	--------------	---------------------	-------------------------

Technology Stack

Exact technologies observed from project manifests, dependency files, and source code.

Layer	Technology	Role in the application
Client application	Flutter, Dart SDK 3.2.5+, Material 3	Cross-platform UI for login, registration, upload selection, expiry configuration, and share-link display.
Client state	flutter_riverpod, shared_preferences	StateNotifier providers manage authentication and upload UI state; the JWT is persisted under auth_token.
Client networking	Dio, file_picker	Dio sends auth and upload requests with an Authorization interceptor; file_picker selects local files for upload.
Backend API	FastAPI, Uvicorn, Pydantic v2	Typed API routes, request/response schemas, form handling, file upload handling, and OpenAPI-compatible service structure.
Database	PostgreSQL 16, SQLAlchemy 2 async, asyncpg	Stores users, file metadata, share-link policy, and download audit records. Binary files are not stored in the database.
Object storage	boto3, MinIO/S3-compatible storage	Stores uploaded chunks and final files. The backend generates S3v4 presigned URLs and supports direct range streaming.
Transient state and jobs	Redis, Celery, Flower	Redis tracks chunked upload sessions and powers Celery broker/result storage; Flower exposes task monitoring.
Edge and deployment	Nginx, Docker Compose, Alembic	Nginx proxies /api, applies request limits, and exposes health. Compose defines local backend infrastructure.
Testing	pytest, pytest-asyncio, flutter_test	Backend route and utility tests use mocked dependencies; Flutter widget tests cover the unauthenticated app state and keyboard layout.

Separation of concerns

The backend keeps route handlers thin. Routes delegate upload, sharing, download, and storage behavior to dedicated service modules.

Metadata versus files

SQL tables track ownership and policy. File bytes live in object storage, reducing database pressure and enabling presigned downloads.

Two upload modes

The backend supports chunked sessions for larger uploads. The current Flutter UI uses the simple multipart endpoint for its visible flow.

Application shell

lib/main.dart wraps the app in ProviderScope, reads authProvider, and selects a loading view, UploadScreen, or LoginScreen. The theme is defined centrally in AppTheme.dark().

- **State:** Riverpod StateNotifier providers expose immutable state objects.
- **Persistence:** shared_preferences stores the bearer token locally.
- **Navigation:** login/register replace the stack with the upload screen after success.

API service

lib/services/api_service.dart creates one Dio client with a base URL from API_BASE_URL at compile time, falling back to a local network IP. An interceptor injects Authorization: Bearer when a saved token exists.

- **Auth calls:** /api/auth/login and /api/auth/register.
- **Upload call:** /api/upload/simple with multipart file and expiry minutes.
- **Error handling:** maps common API status codes to user-facing messages.

Authentication state

AuthNotifier checks for a saved token on startup, performs login/register requests, writes the returned JWT, and clears it on logout. Screens react to provider state.

File transfer state

FileNotifier tracks selected path, loading status, generated link, and expiry minutes. The UI bounds expiry between 10 and 60 minutes, then calls the API service to upload and receive the share URL.

Current client implementation boundary

The backend implements chunked upload endpoints, but the current Flutter upload workflow uses the simple single-file endpoint. This matters for deployment planning because Nginx currently sets client_max_body_size 10M; larger files should use the chunked flow or adjust edge limits.

FastAPI app lifecycle

app.main starts a FastAPI application, applies CORS, mounts /api, verifies database tables through SQLAlchemy metadata, and attempts object-storage bucket initialization during lifespan startup.

Route groups

app.api.router mounts auth, upload, share, and download routers. Each route uses Pydantic schemas and FastAPI dependencies for authenticated user and database session injection.

Visible Flutter upload flow

- 1

User authenticates
The client submits email/password and stores the JWT returned from /api/auth/login.
- 2

User selects file and expiry
file_picker supplies a local path; expiry is constrained to 10-60 minutes.
- 3

Client posts multipart upload
/api/upload/simple uploads bytes to object storage, creates a file row, and creates a share row.
- 4

Share URL returns to UI
The response includes link and expiry_time; the UI displays a copy button.

Backend chunked upload capability

- 1

Start session
/api/upload/start validates size, calculates chunks, creates a Redis hash, and sets TTL.
- 2

Upload chunks
/api/upload/chunk validates owner/index and stores each chunk at an S3 key.
- 3

Finalize file
/api/upload/finalize verifies completeness and assembles the final object using S3 multipart upload.
- 4

Create metadata
The route writes a files record with the final storage key and user ownership.

Share-link policy

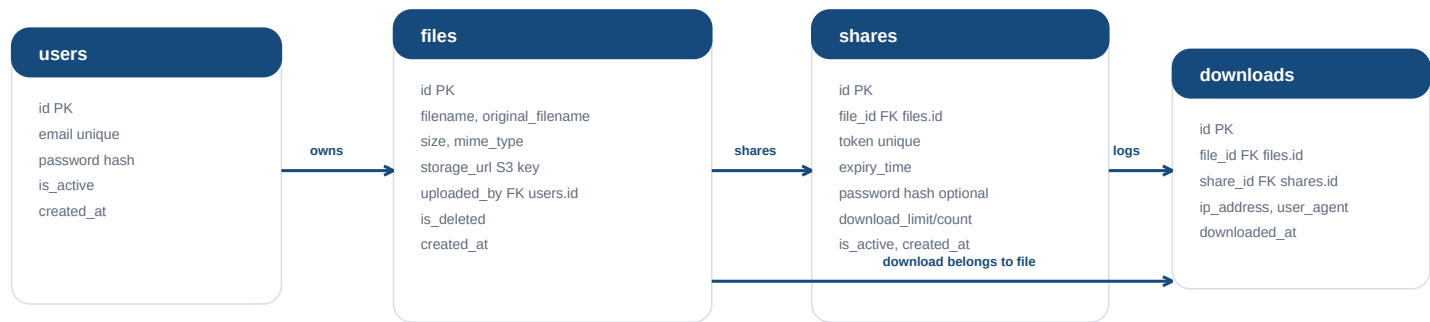
share_service validates file ownership, generates a unique token, computes expiry from minutes or hours, optionally hashes a password, and stores download limits and counters in the shares table.

- Default expiry is configured as 72 hours.
- Minute-based expiry supports 10-60 minutes.
- Protected links use bcrypt-hashed passwords.

Download behavior

download_service validates token state, expiry, password, active flag, and download limit. It then logs the request, increments download count, and either redirects to a presigned object URL or streams the object directly with HTTP range support.

- GET /api/download/{token} returns a 307 redirect.
- GET /api/download/{token}/direct streams bytes.
- HEAD /api/download/{token} exposes size and range support.



Database responsibilities

PostgreSQL stores identities, ownership, object-storage keys, share policy, and audit data. The async SQLAlchemy session dependency commits successful request work and rolls back exceptions.

- **Users:** unique email, password hash, active flag.
- **Files:** original name, generated name, size, MIME type, S3 key, owner.
- **Shares:** token, expiry, optional password, limits, active flag.
- **Downloads:** file/share references plus client IP and user agent.

Object-storage strategy

The storage service uses boto3 against S3 or MinIO. Chunks are stored under uploads/{user_id}/{upload_id}/chunks/{index}. Final files are assembled into uploads/{user_id}/{upload_id}/{filename}.

- Bucket creation is attempted during app startup and before uploads.
- Multipart copy avoids loading every chunk into backend memory.
- Presigned URLs expire after one hour for download redirects.
- Range reads support resumable direct downloads.

Schema migration note

The checked-in Alembic initial migration is currently a placeholder with no generated DDL. At runtime, Base.metadata.create_all verifies and creates tables during FastAPI startup. For production, explicit Alembic migrations should replace startup schema creation.

Security, Reliability, and Operations

Security controls and operational mechanisms directly present in source code.

Security controls

- **Password hashing:** bcrypt via Passlib for user passwords and optional share-link passwords.
- **JWT authentication:** HS256 bearer tokens with configurable expiry, defaulting to 60 minutes.
- **Ownership checks:** upload sessions and share creation validate current-user ownership.
- **Public download gates:** token state, active flag, expiry, password, and download limit are validated.
- **Filename handling:** utility functions sanitize names and generate storage-safe unique identifiers.

Reliability controls

- **Database sessions:** request-scoped async sessions commit, rollback, and close predictably.
- **Redis TTL:** upload sessions expire after the configured TTL and reset on chunk activity.
- **Chunk cleanup:** cancel and scheduled Celery tasks remove stale upload artifacts.
- **Range downloads:** direct streaming supports HTTP Range headers for resume-capable clients.
- **Object-store retries:** boto3 client config enables adaptive retry behavior.

Deployment topology

filesharingbackend/docker-compose.yml defines backend, PostgreSQL, Redis, MinIO, Nginx, Celery worker, and Flower services on a bridge network. The backend is served by Uvicorn on port 8000.

- Backend: 8000
- Nginx: 80
- PostgreSQL: 5432
- Redis: 6379
- MinIO API/console: 9000 / 9001
- Flower: 5555

Nginx edge behavior

nginx.conf proxies /api/ to the backend, forwards client headers, exposes /health, applies 100r/m per-IP rate limiting, and configures extended proxy timeouts for file-transfer requests.

- client_max_body_size 10M is compatible with 5 MB chunks plus buffer.
- Rate limit burst is set to 20 requests with no delay.
- Proxy read/send timeout is 300 seconds for large transfers.

Background processing

Celery uses Redis as broker and result backend. Scheduled tasks deactivate expired share links every hour and clean up nearly expired in-progress upload sessions every 30 minutes.

Environment configuration

pydantic-settings centralizes app, database, Redis, object storage, file-size, expiry, and CDN settings. Sensitive values such as secret keys and access keys are expected from environment variables.

Implementation Notes and Source Map

Precise boundaries, known considerations, and source evidence used for this report.

Current UI is focused, not administrative.

The Flutter screens expose authentication, file selection, expiry selection, upload, and copyable link output. Backend routes include additional share listing, file listing, revoke, progress, cancel, and direct-download capabilities.

Object storage is S3-compatible, with MinIO configured locally.

Dependencies include boto3/botocore, and Compose includes a MinIO service. The code can target MinIO through MINIO_ENDPOINT or generic S3-style storage when configured differently.

Testing exists but is primarily mocked at the API boundary.

Backend route tests patch service calls and dependency overrides. Flutter widget tests check initial unauthenticated rendering and keyboard behavior.

Documentation source is the codebase.

README.md is empty, so this PDF is based on direct inspection of implementation files, manifests, and tests.

Source file	Evidence used
pubspec.yaml	Flutter dependencies: Dio, Riverpod, file_picker, shared_preferences, Google Fonts, and lints.
lib/main.dart, lib/providers/*, lib/services/api_service.dart	Client routing, auth persistence, API base URL, bearer-token interceptor, upload flow, and state model.
filesharingbackend/backend/app/main.py	FastAPI initialization, lifespan startup, CORS policy, router mounting, and health endpoint.
app/api/routes/*.py	Auth, upload, share, and download endpoint contracts and HTTP behavior.
app/services/*.py	Upload sessions, chunk assembly, S3/MinIO operations, share validation, presigned URLs, and direct streaming.
app/models/*.py	SQLAlchemy entities for users, files, shares, and downloads.
docker-compose.yml, nginx/nginx.conf	Container topology, service ports, dependency order, reverse proxy, rate limiting, and upload body limit.
backend/tests/*, test/widget_test.dart	Backend and Flutter test coverage areas.

Prepared as a professional technical architecture PDF for the Naiyo24 Transfer repository. No assumptions were taken from external documentation.